



第3章 单片机汇编语言程序设计

刘剑锋

ljf@csu.edu.cn





主要内容

3. 1 51系列单片机指令系统概述

3. 2 51系列单片机寻址方式

3. 3 51系列单片机指令系统

3. 4 51系列单片机汇编指令格式及标识

3. 5 51系列单片机汇编程序常用伪指令

3. 6 51系列单片机汇编程序设计

§ 3.1 51系列单片机指令系统概述

指令：计算机可以直接识别和执行的某种操作命令

指令系统：计算机中所有指令的集合。

一条指令的结构用如下形式表示：

操作码字段OP	地址码字段A
---------	--------

操作码字段表征指令的操作特性与功能

地址码字段通常指定参与操作的操作数的地址

§ 3.1 51系列单片机指令系统概述

在51单片机中的指令系统中，有**单字节**、**双字节**、**三字节**等不同长度的指令。

- **单字节指令**只有一个字节，操作码和操作数都在这个字节中。51指令系统共有单字节指令49条。
- **双字节指令**包括两个字节，其中一个字节是操作码，第二个字节是操作数。51指令系统共有双字节指令45条。
- **三字节指令**中，操作码占一个字节，操作数占两个字节，其中操作数既可能是数据，也可能是地址。51指令系统共有三字节指令45条。

§ 3.2 51系列单片机寻址方式

MCS-51单片机的寻址方式按操作数的类型，可分为**数的寻址**和**指令寻址**。

数的寻址有：

- 常数寻址（立即寻址）、
- 寄存器数寻址（寄存器寻址）、
- 存储器数寻址（直接寻址方式、寄存器间接寻址方式、变址寻址方式）
- 位寻址。

指令的寻址有：绝对寻址和相对寻址。

§ 3.2 51系列单片机寻址方式

3.2.1 立即寻址

操作数是常数，常数又称为立即数，使用时直接出现在指令中，紧跟在操作码的后面，作为指令的一部分。

在汇编指令中，立即数前面以“#”符号作前缀。
在程序中通常用于给寄存器或存储器单元赋初值，

例如：

```
MOV A, #20H
```

§ 3.2 51系列单片机寻址方式

3.2.2 寄存器寻址

操作数在寄存器中，使用时在指令中直接提供寄存器的名称，这种寻址方式称为寄存器寻址。

在汇编指令中，寄存器寻址在指令中直接提供寄存器的名称，如R0、R1、A、DPTR等。

例如：

MOV A, R0

§ 3.2 51系列单片机寻址方式

3.2.3 直接寻址

直接寻址是指数据在存储器单元中，在指令中直接提供存储器单元的地址。

这种寻址方式针对的是片内数据存储器 and 特殊功能寄存器。

直接寻址方式的寻址范围包括：

1. 低128单元，在指令中直接以单元地址形式给出。

例如：MOV A, 20H ; 把片内数据存储器20H单元的内容送给累加器A。

2. 专用寄存器，以单元地址形式或寄存器符号形式给出。

例如：MOV A, SBUF ; SBUF是专用寄存器名。

✓ 直接寻址是访问专用寄存器的唯一方法。

§ 3.2 51系列单片机寻址方式

3.2.4 寄存器间接寻址

寄存器间接寻址是指数据存放在存储单元中，而存储单元地址存放在寄存器中，在指令中通过提供存放存储单元地址的寄存器来使用对应的存储单元。

形式为：@寄存器名

例如：MOV A, @R1

该指令的功能是将以工作寄存器R1中的内容为地址的片内RAM单元的数据传送到累加器A中去。

§ 3.2 51系列单片机寻址方式

- 寄存器间接寻址用到的寄存器只能是通用寄存器R0、R1和数据指针寄存器DPTR，它能访问的数据是片内数据存储器 and 片外数据存储器。
- 其中，片内数据存储器只能用R0和R1作指针间接访问；
- 片外数据存储器，低端的256字节单元，既可以用两位十六进制地址以R0或R1作指针间接访问，也可用四位十六进制地址以DPTR作指针间接访问。而高端的字节单元则只能以DPTR作指针间接访问。
- 对于片内RAM和片外RAM的低端256字节都可以用R0和R1作指针访问，它们之间用指令来区别。片内RAM访问用MOV指令，片外RAM访问用MOVB指令。

§ 3.2 51系列单片机寻址方式

3.2.5 相对寻址

相对寻址是把指令中给定的地址偏移量与程序计数器 PC 的当前值相加，得到真正的程序转移地址。相对寻址方式主要是解决程序转移而专门设置的。

例如：JC 80H

若C=0，则PC值不变，若C=1，则以当前PC值为基地址，加上偏移量80H得到新的PC值。

这里的**偏移量**是一个带符号的二进制补码数，所能表示的数的范围是-128~+127。

因此相对转移是以转移指令的下条指令为基点，向前最大可能转移127个字节，向后最大可能转移128个字节。

§ 3.2 51系列单片机寻址方式

3.2.6 变址寻址

变址寻址是指操作数的地址由基址寄存器的地址加上变址寄存器的地址得到。

它是以数据指针寄存器DPTR或程序计数器PC为基址，累加器A为变址，两者相加得存储单元的地址，所访问的存储器为程序存储器。

MOVC A, @A+DPTR

MOVC A, @A+PC

§ 3.2 51系列单片机寻址方式

3.2.7 位寻址

51指令系统有位处理功能，可以对数据位进行操作，因此就有相应的位寻址方式。

位寻址的范围包括：

(1) 内部RAM中的位寻址区。单元地址为20H~2FH，共16个单元128位，位地址是00H~7FH。

(2) 专用寄存器的可寻址位。可以位寻址的专用寄存器共有11个，实有寻址位83位。

§ 3.3 51系列单片机指令系统

3.3.1 数据传送指令

数据传送指令属于复制，传送后，源操作数不变，不影响别的寄存器和标志。

传送指令包括“MOV”、“MOVB”、“MOVC”。

数据传送指令的约定是从右向左，左边是目的操作数，右边是源操作数。

源操作数可以是：累加寄存器、通用寄存器、直接地址、间接地址和立即数。

目的操作数除了不能用立即数，其它和源操作数一样。

§ 3.3 51系列单片机指令系统

3.3.1.1 片内数据存储器传送指令 MOV

指令格式：

MOV 目的操作数，源操作数

其中：

源操作数可以为A、Rn、@Ri、direct、#data

目的操作数可以为A、Rn、@Ri、direct

组合起来总共16条，按目的操作数的寻址方式划分为五组：

§ 3.3 51系列单片机指令系统

(1) 以A为目的操作数

```
MOV A, Rn ; A ← (Rn)
MOV A, direct ; A ← (direct)
MOV A, @Ri ; A ← ((Ri))
MOV A, #data ; A ← #data
```

(2) 以Rn为目的操作数

```
MOV Rn, A ; Rn ← (A)
MOV Rn, direct ; Rn ← (direct)
MOV Rn, #data ; Rn ← #data
```

(3) 以直接地址direct为目的操作数

```
MOV direct, A ; direct ← (A)
MOV direct, Rn ; direct ← (Rn)
MOV direct, direct ; direct ← (direct)
MOV direct, @Ri ; direct ← ((Ri))
MOV direct, #data ; direct ← #data
```

(4) 以间接地址@Ri为目的操作数

```
MOV @Ri, A ; (Ri) ← (A)
MOV @Ri, direct ; (Ri) ← (direct)
MOV @Ri, #data ; (Ri) ← #data
```

(5) 以DPTR为目的操作数

```
MOV DPTR, #data16 ; DPTR ← #data16
```

MOV Rn, Rn

×

MOV @Ri, Rn

×

注意：

- 源操作数和目的操作数中的Rn和@Ri不能相互配对。
- 在MOV指令中，不允许在一条指令中同时出现工作寄存器，无论它是寄存器寻址还是寄存器间接寻址。

§ 3.3 51系列单片机指令系统

3.3.1.2 片外数据存储器传送指令MOVX

在MCS-51系统中只能通过累加器A与片外数据存储器进行数据传送，访问时只能通过@Ri和@DPTR以间接寻址方式进行。

MOVX指令共有四条：

MOVX A, @DPTR ; $A \leftarrow (DPTR)$

MOVX @DPTR, A ; $(DPTR) \leftarrow (A)$

MOVX A, @Ri ; $A \leftarrow (Ri)$

MOVX @Ri, A ; $(Ri) \leftarrow (A)$

其中前两条指令通过DPTR间接寻址，可以对整个64K片外数据存储器访问。

后两条指令通过@Ri间接寻址，只能对片外数据存储器的低端的256字节访问，访问时将低8位地址放于Ri中。

§ 3.3 51系列单片机指令系统

3.3.1.3 程序存储器数据传送指令MOVC

程序存储器传送指令只有两条，一条用DPTR基址变址寻址。一条用PC基址变址寻址。

MOVC A, @A+DPTR ; $A \leftarrow (A+DPTR)$

MOVC A, @A+PC ; $A \leftarrow (A+PC)$

这两条指令通常用于访问表格数据，因此称为查表指令。

§ 3.3 51系列单片机指令系统

【例3-1】 写出完成下列功能的程序段。

(1) 将R0的内容送R6中

```
MOV  A, R0  
MOV  R6, A
```

(2) 将片内RAM 30H单元的内容送片外60H单元中

```
MOV  A, 30H  
MOV  R0, #60H  
MOVX @R0, A
```

(3) 将片外RAM 1000H单元的内容送片内20H单元中

```
MOV  DPTR, #1000H  
MOV  A, @DPTR  
MOV  20H, A
```

(4) 将ROM 2000H单元的内容送片内RAM的30H单元中

```
MOV  A, #0  
MOV  DPTR, #2000H  
MOVC A, @A+DPTR  
MOV  30H, A
```

§ 3.3 51系列单片机指令系统

3.3.1.4 数据交换指令

数据交换指令要求第一个操作数须为累加器A，共有5条。

```
XCH      A, Rn      ; A<=> Rn
XCH      A , direct  ; A<=>direct
XCH      A, @Ri      ; A<=> (Ri)
XCHD     A, @Ri      ; A0~3<=> (Ri)0~3
SWAP     A           ; A0~3<=>A4~7
```

【例】若R0的内容为30H，片内RAM 30H单元的内容为23H，累加器A的内容为45H。

执行XCH A, @R0 指令后片内RAM 30H单元的内容为45H，累加器A中的内容为23H。

执行SWAP A 指令，则累加器A的内容为54H。

§ 3.3 51系列单片机指令系统

3.3.1.5 堆栈指令

堆栈是在片内RAM中按“先进后出，后进先出”原则设置的专用存储区。数据的进栈和出栈由指针SP统一管理。

堆栈操作指令有两条：

`PUSH direct ; SP ← (SP+1), (SP) ← (direct)`

`POP direct ; direct ← (SP), (SP) ← (SP-1)`

其中PUSH指令入栈，POP指令出栈。

操作时以字节为单位。

§ 3.3 51系列单片机指令系统

3.3.2 算术运算指令

3.3.2.1 加法指令

1. 一般的加法指令ADD

ADD A, Rn ; $A \leftarrow (A) + (Rn)$
ADD A, direct ; $A \leftarrow (A) + (\text{direct})$
ADD A, @Ri ; $A \leftarrow (A) + ((Ri))$
ADD A, #data ; $A \leftarrow (A) + \#data$

2. 带进位加指令ADDC

ADDC A, Rn ; $A \leftarrow (A) + (Rn) + C$
ADDC A, direct ; $A \leftarrow (A) + (\text{direct}) + C$
ADDC A, @Ri ; $A \leftarrow (A) + (Ri) + C$
ADDC A, #data ; $A \leftarrow (A) + \#data + C$

3. 加1指令

INC A ; $A \leftarrow (A) + 1$
INC Rn ; $Rn \leftarrow (Rn) + 1$
INC direct ; $\text{direct} \leftarrow (\text{direct}) + 1$
INC @Ri ; $(Ri) \leftarrow ((Ri)) + 1$
INC DPTR ; $DPTR \leftarrow (DPTR) + 1$

其中:

- ADD和ADDC指令在执行时要影响CY、AC、OV和P标志位。
- 而INC指令除了INC A要影响P标志位外, 对其它标志位都没有影响。

§ 3.3 51系列单片机指令系统

3.3.2 算术运算指令

3.3.2.2 减法指令

1. 带借位减法指令SUBB

SUBB A, Rn	; $A \leftarrow (A) - (Rn) - C$
SUBB A, direct	; $A \leftarrow (A) - (\text{direct}) - C$
SUBB A, @Ri	; $A \leftarrow (A) - (Ri) - C$
SUBB A, #data	; $A \leftarrow (A) - \#data - C$

2. 减1指令DEC

DEC A	; $A \leftarrow (A) - 1$
DEC Rn	; $Rn \leftarrow (Rn) - 1$
DEC direct	; $\text{direct} \leftarrow (\text{direct}) - 1$
DEC @Ri	; $(Ri) \leftarrow (Ri) - 1$

在51单片机中，只提供了一种带借位的减法指令，没有提供一般的减法指令，一般的减法操作可以通过先对CY标志清零，然后再执行带借位的减法来实现。

§ 3.3 51系列单片机指令系统

3.3.2 算术运算指令

3.3.2.3 乘法指令

在MCS-51单片机中，乘法指令只有一条：

MUL AB

该指令执行时将存放于累加器A中的无符号被乘数和放于B寄存器中的无符号乘数相乘，积的高字节存于B寄存器中，低字节存于累加器A中。

指令执行后将影响CY和OV标志。

3.3.2.3 除法指令

在MCS-51单片机中，除法指令也只有一条：

DIV AB

该指令执行时将存放在累加器A中的无符号被除数与存放在B寄存器中的无符号除数相除，除得的结果，**商**存累加器A，**余数**存B寄存器。

指令执行后将影响CY和OV标志。

§ 3.3 51系列单片机指令系统

3.3.2 算术运算指令

3.3.2.5 十进制调整指令

MCS-51单片机中，十进制调整指令只有一条：

DA A

可实现两位十进制BCD码数的加法运算。它的调整过程为：

- (1) 若累加器A的低四位为十六进制的A-F或辅助进位标志AC为1，则累加器A中的内容作加06H调整。
- (2) 若累加器A的高四位为十六进制的A-F或进位标志CY为1，则累加器A中的内容作加60H调整

【例】在R3中有十进制数67，在R2中有十进制数85，用十进制运算，运算的结果放于R5中。

程序为：

```
MOV  A, R3
ADD  A, R2
DA   A
MOV  R5, A
```

§ 3.3 51系列单片机指令系统

3.3.3 逻辑运算指令

3.3.3.1 逻辑与指令ANL

```
ANL A, Rn      ; A ← (A) ∧ (Rn)
ANL A, direct  ; A ← (A) ∧ (direct)
ANL A, @Ri     ; A ← (A) ∧ ( (Ri) )
ANL A, #data   ; A ← (A) ∧ #data
ANL direct, A  ; direct ← (direct) ∧ (A)
ANL direct, #data ; direct ← (direct) ∧ #data
```

3.3.3.2 逻辑或指令ORL

```
ORL A, Rn      A ← (A) ∨ (Rn)
ORL A, direct  A ← (A) ∨ (direct)
ORL A, @Ri     A ← (A) ∨ ( (Ri) )
ORL A, #data   A ← (A) ∨ #data
ORL direct, A  direct ← (direct) ∨ (A)
ORL direct, #data direct ← (direct) ∨ #data
```

3.3.3.3 逻辑异或指令XRL

```
XRL A, Rn      A ← (A) ∨ (Rn)
XRL A, direct  A ← (A) ∨ (direct)
XRL A, @Ri     A ← (A) ∨ ( (Ri) )
XRL A, #data   A ← (A) ∨ #data
XRL direct, A  direct ← (direct) ∨ (A)
XRL direct, #data direct ← (direct) ∨ #data
```

3.3.3.4 清零和求反指令

清零指令: CLR A ; A ← 0

求反指令: CPL A ; A ← $\bar{A}$

在MCS-51系统中, 只能对累加器A中的内容进行清零和求反, 如要对其它的寄存器或存储单元进行清零和求反, 则须放在累加器A进行, 运算后再放回原位置。

3.3.3.5 循环移位指令

MCS-51系统有四条对累加器A的循环移位指令, 前两条只在累加器A中进行循环移位, 后两条还要带进位标志CY进行循环移位, 每一次移一位。分别如下:

1. 累加器A循环左移

RL A

2. 累加器A循环右移

RR A

3. 带进位的循环左移

RLC A

4. 带进位的循环右移

RRC A

§ 3.3 51系列单片机指令系统

3.3.4 控制转移指令

控制转移指令分为：无条件转移指令、有条件转移指令和子程序调用和返回指令。

3.3.4.1 无条件转移指令

无条件转移指令是指当执行该指令后，程序将无条件地转移到指令指定的地方去。无条件转移指令包括长转移指令、绝对转移指令、相对转移指令和间接转移指令。

§ 3.3 51系列单片机指令系统

3.3.4 控制转移指令

分为4种：无条件转移指令、有条件转移指令、子程序调用、返回指令。

3.3.4.1 无条件转移指令

无条件转移指令是指当执行该指令后，程序将无条件地转移到指令指定的地方去。

1. 长转移指令 LJMP

LJMP addr16 ; PC $\leftarrow$ addr16

2. 绝对转移指令 AJMP

AJMP addr11 ; PC_{10~0} $\leftarrow$ addr11

3. 短转移指令 SJMP

SJMP rel ; PC $\leftarrow$ (PC) + 2 + rel

4. 变址寻址转移指令 JMP

JMP @A+DPTR ; PC $\leftarrow$ (A) + (DPTR)

§ 3.3 51系列单片机指令系统

3.3.4.2 有条件转移指令

有条件转移指令是指当条件满足时，程序转移到指定位置，条件不满足时，程序将继续顺次执行。

1. 累加器A判零条件转移指令

- ① **JZ rel** ; 若A=0, 则PC $\leftarrow$ (PC) + 2 + rel, 否则, PC $\leftarrow$ (PC) + 2
- ② **JNZ rel** ; 若A $\neq$ 0, 则PC $\leftarrow$ (PC) + 2 + rel, 否则, PC $\leftarrow$ (PC) + 2

2. 比较不等转移指令

- ① **CJNE A, #data, rel** ; 若A=data, 则PC $\leftarrow$ (PC) + 3, 不转移, 继续执行。若A>data, 则C=0, PC $\leftarrow$ (PC) + 3 + rel, 转移。若A<data, 则C=1, PC $\leftarrow$ (PC) + 3 + rel, 转移。
- ② **CJNE Rn, #data, rel** ; 若(Rn)=data, 则PC $\leftarrow$ (PC) + 3, 不转移, 继续执行。若(Rn)>data, 则C=0, PC $\leftarrow$ (PC) + 3 + rel, 转移。若(Rn)<data, 则C=1, PC $\leftarrow$ (PC) + 3 + rel, 转移。
- ③ **CJNE @Ri, #data, rel** ; 若((Ri))=data, 则PC $\leftarrow$ (PC) + 3, 不转移, 继续执行。若((Ri))>data, 则C=0, PC $\leftarrow$ (PC) + 3 + rel, 转移。若((Ri))<data, 则C=1, PC $\leftarrow$ (PC) + 3 + rel, 转移。
- ④ **CJNE A, direct, rel** ; 若A=direct, 则PC $\leftarrow$ (PC) + 3, 不转移, 继续执行。若A>direct, 则C=0, PC $\leftarrow$ (PC) + 3 + rel, 转移。若A<direct, 则C=1, PC $\leftarrow$ (PC) + 3 + rel, 转移。

3. 循环减1转移指令

- ① **DJNZ Rn, rel** ; 先Rn中的内容减1, 再判断Rn中的内容是否等于零, 若不为零, 则转移。
- ② **DJNZ direct, rel** ; 先(direct)中的内容减1, 再判断(direct)中的内容是否等于零, 若不为零, 则转移。

4. 判C转移指令

- ① **JC rel** ; 若C=1, 则转移, PC $\leftarrow$ (PC)+2+rel ; 否则程序继续执行
- ② **JNC rel** ; 若C=0, 则转移, PC $\leftarrow$ (PC)+2+rel ; 否则程序继续执行

5. 判直接寻址位转移指令

- ① **JB bit, rel** ; 若(bit)=1, 则转移, PC $\leftarrow$ (PC)+3+rel; 否则程序继续执行
- ② **JNB bit, rel** ; 若(bit)=0, 则转移, PC $\leftarrow$ (PC)+3+rel; 否则程序继续执行
- ③ **JBC bit, rel** ; 若(bit)=1, 则转移, PC $\leftarrow$ (PC)+3+rel, 且(bit) $\leftarrow$ 0; 否则程序继续执行

§ 3.3 51系列单片机指令系统

3.3.4.3 子程序调用指令

1. 绝对调用指令

指令格式: ACALL addr11

执行过程: $PC \leftarrow (PC) + 2$

$SP \leftarrow (SP) + 1$

$(SP) \leftarrow (PC)_{7 \sim 0}$

$SP \leftarrow (SP) + 1$

$(SP) \leftarrow (PC)_{15 \sim 8}$

$PC_{10 \sim 0} \leftarrow \text{addr11}$

2. 长调用指令

指令格式: LCALL addr16

执行过程: $PC \leftarrow (PC) + 3$

$SP \leftarrow (SP) + 1$

$(SP) \leftarrow (PC)_{7 \sim 0}$

$SP \leftarrow (SP) + 1$

$(SP) \leftarrow (PC)_{15 \sim 8}$

$PC \leftarrow \text{addr16}$

3. 子程序返回指令

指令格式: RET

执行过程: $PC_{15 \sim 8} \leftarrow ((SP))$

$SP \leftarrow (SP) - 1$

$PC_{7 \sim 0} \leftarrow ((SP))$

$SP \leftarrow (SP) - 1$

4. 中断子程序返回指令

指令格式: RETI

执行过程: $PC_{15 \sim 8} \leftarrow ((SP))$

$SP \leftarrow (SP) - 1$

$PC_{7 \sim 0} \leftarrow ((SP))$

$SP \leftarrow (SP) - 1$

3.3.4.4 空操作指令

指令格式: NOP ; $PC \leftarrow$

$(PC) + 1$

§ 3.3 51系列单片机指令系统

3.3.5 位操作指令

3.3.5.1 位数据传送指令

位传送指令有2条:

```
MOV C, bit ; C ← (bit)
```

```
MOV bit, C ; (bit) ← C
```

指令在使用时必须有位运算器C参与，不能直接实现两位之间的传送。

如果进行两位之间的传送，可以通过位运算器C来传送。

例：把片内RAM中位寻址区的20H位的内容传送到30H位。

```
MOV C, 20H
```

```
MOV 30H, C
```

3.3.5.2 位数据清零指令

```
CLR C ; C ← 0
```

```
CLR bit ; (bit) ← 0
```

3.3.5.3 位数据置1指令

```
SETB C ; C ← 1
```

```
SETB bit ; (bit) ← 1
```

3.3.5.4 位数据取反指令

```
CPL C ; C ← /C
```

```
CPL bit ; (bit) ← (bit)
```

3.3.5.5 位数据与指令

```
ANL C, bit ; C ← C ∧ (bit)
```

```
ANL C, /bit ; C ← C ∧ (/bit)
```

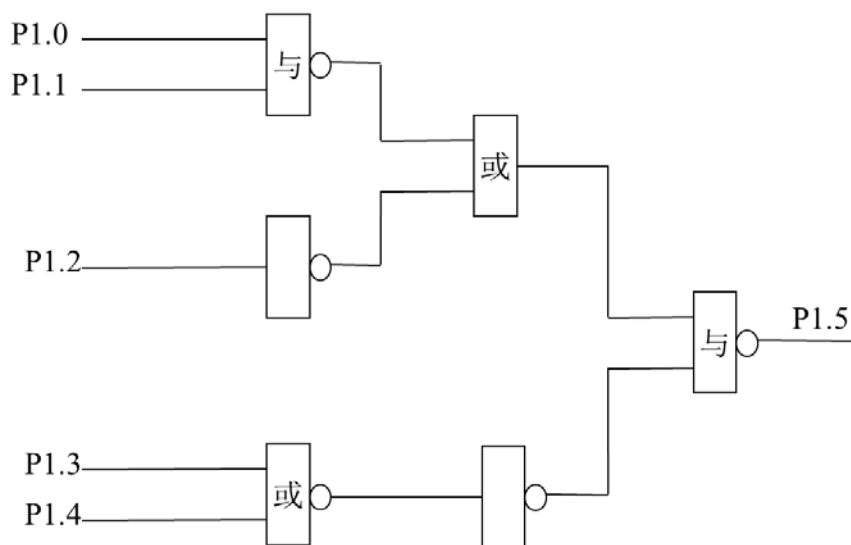
3.3.5.6 位数据或指令

```
ORL C, bit ; C ← C ∨ (bit)
```

```
ORL C, /bit ; C ← C ∨ (/bit)
```


§ 3.3 51系列单片机指令系统

【例】利用位逻辑运算指令编程实现下面硬件逻辑电路的功能。



程序：

```
MOV    C, P1.0
ANL     C, P1.1
CPL     C
ORL     C, /P1.2
MOV     F0, C
MOV     C, P1.3
ORL     C, P1.4
ANL     C, F0
CPL     C
MOV     P1.5, C
```


§ 3.4 51系列单片机汇编指令格式

[标号:] 操作码助记符 [目的操作数][, 源操作数] [; 注释]

- (1) 操作码助记符表明指令的功能，不同的指令有不同的指令助记符，它一般用说明其功能的英文单词的缩写形式表示。
- (2) 操作数用于给指令的操作提供数据、数据的地址或指令的地址，操作数往往用相应的寻址方式指明。
- (3) 标号是该指令的符号地址，后面须带冒号。
- (4) 注释是对该指令的解释，前面须带分号。

§ 3.5 51系列单片机汇编程序常用伪指令

3.5.1 定义起始地址伪指令

格式: ORG 地址 (十六进制表示)

【例】 ORG 1000H

START: MOV A, #7FH

指明后面的程序从程序存储器的1000H单元开始存放。

3.5.2 定义汇编结束伪指令

格式: END

该指令放于程序最后位置, 用于指明汇编语言源程序的结束位置, 当汇编程序汇编到END伪指令时, 汇编结束。

3.5.3 标号赋值伪指令

格式: 符号 EQU 项

【例】 TAB1 EQU 1000H

TAB2 EQU 2000H

§ 3.5 51系列单片机汇编程序常用伪指令

3.5.4 字节定义伪指令

【例3-20】

格式: [标号:] DB 项或项表

```
ORG 3000H
TAB1: DB 12H, 34H
      DB '5', 'A',
      'abc'
```

3000H	12H
3001H	34H
3002H	35H
3003H	41H
3004H	61H
3005H	62H
3006H	63H
3007H	-
3008H	-
3009H	-
300AH	-
300BH	56H
300CH	78H

3.5.5 字定义伪指令

格式: [标号:] DW 项或项表

```
DS 4H
TAB2: DW 5678H
```

3.5.6 存储区定义伪指令

格式: [标号:] DS 数值表达

【例3-23】

```
FLG bit F0
AI bit P1.0
定义后, 在程序中位地址F0、
P1.0就可以通过FLG和AI来使用
```

式

3.5.7 位定义伪指令

格式: 符号 bit 位地址。

§ 3.5 51系列单片机汇编程序常用伪指令

3.5.8 内部RAM地址赋值伪指令

格式：符号 DATA 直接字节地址

该伪指令用于给片内RAM字节单元地址赋予DATA前面的符号，赋值后可用该符号代替DATA后面的片内RAM字节单元地址。

【例3-24】

```
RESULT DATA 60H  
MOV RESULT, A
```

3.5.9 外部RAM地址赋值伪指令

格式：符号 XDATA 直接字节地址

该伪指令与DATA伪指令基本相同，只是它针对的是片外RAM字节单元。

【例3-25】

```
PORT1 XDATA 2000H  
MOV DPTR, PORT1  
MOVX @DPTR
```

§ 3.6 51系列单片机汇编程序设计

汇编程序的设计常采用以下几种基本结构：

①顺序结构

②分支结构

③循环结构

④子程序

⑤中断服务子程序

➤ 任何复杂的程序都是由这几种基本结构复合而成的

3.6.1 顺序结构程序设计

例3-26：已知X、Y、Z分别为片内RAM 30H、31H、32H单元的内容，设 $X > Y$ ，试编程完成下式的算术运算： $S = (X - Y) * Z$ ，并将计算结果S存入片内RAM 34H(高字节)、35H(低字节)单元中。

解：由于 $X > Y$ ，所以 $X - Y > 0$ ，不需要借位； $(X - Y) * Z$ 的结果最多占用两个字节。首先将 $X - Y$ 结果放入A寄存器中，将Z的内容放入B寄存器中，然后将A和B相乘，其结果的高低字节分别置于B和A寄存器中，然后将它们分别放入34H和35H存储单元中。

程序如下：

```
ORG      0030H

START: MOV    A, 30H    ;A←X

        CLR C

        SUBB   A, 31H    ;A←(A)-Y

        MOV    B, 32H    ;B←Z

        MUL    AB

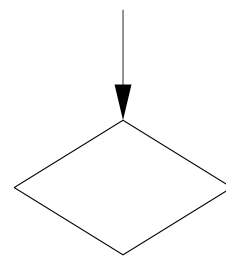
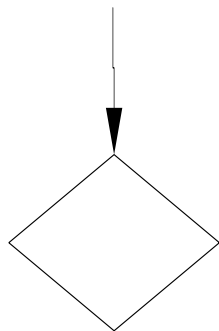
        MOV    34H, B    ;34H←A*B的高字节

        MOV    35H, A    ;35H←A*B的低字节

        SJMP   $
```

3.6.2 分支结构设计

分支结构可以有两种形式，单分支结构和多分支结构。



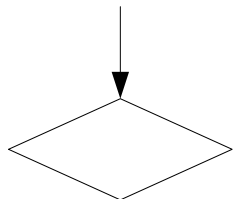
例3-27：假设外部RAM中有三个8位无符号数据，它们的地址分别为7111H、5687H和8766H，找出它们之中最大的数放入寄存器A中。

解：先将地址为7111H、5687H的两个数相比较，将其中较大的数放置于R0。然后将R0和地址为8766H的数相比较，将其中较大数置于A中。

```
ORG      0000H
MOV  DPTR, #7111H
MOVX A, @DPTR
MOV  R0, A
MOV  DPTR, #5687H
MOVX A, @DPTR
MOV  R1, A
CLR  C
SUBB A, R0
JC   BIG1          ;      R0>R1转
BIG1:
MOV  A, R1
MOV  R0, A
BIG2:MOV  A, R0
LABLAST: SJMP $
END
```


例3-28：寄存器A中存放了一个成绩值（ $0 \leq A \leq 100$ ），编写一程序，判定该成绩所属的级别，放入寄存器B中。判定标准是：1级（ $100 \leq A \leq 90$ ）、2级（ $89 \leq A \leq 80$ ）、3级（ $79 \leq A \leq 70$ ）、4级（ $69 \leq A \leq 60$ ）、5级（ $A \leq 59$ ）。

解：按照题意, 该程序的流程图如下：



3.6.5 子程序设计

调用子程序的指令是 ACALL 和 LCALL。

子程序的调用需要注意以下几点：

保存与恢复寄存器：由于主程序和子程序经常是分别编写的，所以它们所使用的寄存器会发生冲突。主程序在调用子程序以前，要保护那些可能被子程序破坏的寄存器。在结束子程序运行前，要恢复那些被保护的寄存器。一般采用压栈和出栈的方法实现寄存器的保护和恢复。

子程序的参数传送：主程序在调用子程序时，经常需要传送一些参数给子程序，成为输入参数；子程序运行完后经常要回送一些信息给主程序，称为输出参数。

3.6.5 子程序设计

例3-32：已知寄存器R0、R1、R2存放的是八位二进制数，分别将它们转换成BCD码，存放在内部RAM的60H~65H单元。八位二进制数的对应BCD码是3位（百位、十位、个位）十进制数，总共12个比特位，所以用两个字节来表示对应BCD码。十位和个位分别存放在第一个字节的高4位和低4位，百位存放在第二个字节的低四位。第二个字节的高四位为全0。

解：编写一个子程序，该子程序的功能是转换一个八位二进制数为BCD码。

该子程序的输入参数是：待转换的单字节数在累加器A中。

输出参数是：转换后的BCD码整数(十位和个位)仍在累加器A中，百位在R3中。

根据题意，将该子程序调用三次，每次的结果放置到相应的内存单元中。

3.6.5 子程序设计

ORG 0000H

MOV A, R0

ACALL CHBCD

MOV 60H, A ;将十位数和个位数置于存储器中

MOV 61H, R3 ;将百位数置于存储器中

MOV A, R1

ACALL CHBCD

MOV 62H, A ;将十位数和个位数置于存储器中

MOV 63H, R3 ;将百位数置于存储器中

MOV A, R2

ACALL CHBCD

MOV 64H, A ;将十位数和个位数置于存储器中

MOV 65H, R3 ;将百位数置于存储器中

3.6.5 子程序设计

CHBCD: **PUSH PSW** ;保护现场

PUSH B

MOV B, #100

DIV AB ;A←BCD码的百位

MOV R3, A ;R3←BCD码的百位

MOV A, #10 ;余数继续分离十位和个位

XCH A, B

DIV AB ;A←BCD码的十位

SWAP A

ORL A, B ;将十位和个位拼装成BCD码

POP B ;恢复现场

POP PSW

RET

END



本章小结